

SGT UNIVERSITY

School of Engineering and Technology

Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 4

SELECT Queries — DISTINCT, WHERE, BETWEEN, IN, ORDER BY, UNION, INTERSECT

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	09 September 2026
Dataset	employee (10 rows); cse and mechanical (5 students each, 2 shared)
Methods	SELECT, DISTINCT, WHERE, BETWEEN, IN, ORDER BY, UNION, UNION ALL, INTERSECT

1. Aim

To answer the lab questions below: create an employee table with the given attributes, insert 10 rows, and run the listed SELECT queries (DISTINCT, WHERE, BETWEEN, IN, ORDER BY). Then create compatible cse and mechanical student tables and combine them with UNION, UNION ALL, and INTERSECT.

2. Theory

2.1 SELECT

SELECT reads rows from a relation. SELECT * returns every column; naming columns returns only those attributes. A query without a WHERE clause returns every row that currently exists in the table.

2.2 DISTINCT

SELECT DISTINCT removes duplicate values from the result. Question 3 on the lab sheet asks for a “distant” department number; that is DISTINCT. SELECT department_no lists a department once per employee (question 4); SELECT DISTINCT department_no lists each department number only once.

2.3 WHERE

The WHERE clause keeps rows that satisfy a condition. Comparison operators such as > and <> filter numbers and strings. OR keeps a row if either predicate is true (city is Delhi or Mumbai). City values in this lab are stored in title case and matched with the same casing so the filters work in SQLite without extra case-conversion functions.

2.4 BETWEEN

BETWEEN low AND high is an inclusive range: the endpoints are part of the result. NOT BETWEEN is the complement — salaries strictly below 30000 or strictly above 50000. It is equivalent to salary < 30000 OR salary > 50000 when the range is 30000 to 50000.

2.5 IN

IN (...) tests membership in a list. city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore') keeps those four cities. NOT IN keeps every city that is not in the list — here Bangalore, Hyderabad, and Pune remain when Mumbai, Delhi, and Chennai are excluded.

2.6 ORDER BY

ORDER BY sorts the result. ASC (the default) lists names A→Z; DESC lists them Z→A. Sorting does not change stored rows; it only changes the order of the result set.

2.7 UNION

UNION stacks two compatible SELECT results and keeps each distinct row once. Both sides must return the same number of columns with comparable types. Duplicate rows that appear in both cse and mechanical are listed only once. Column names in the result come from the first SELECT.

2.8 UNION ALL

UNION ALL also stacks two results but does **not** remove duplicates. A row that exists in both tables appears twice. Use it when every occurrence matters, or when the extra distinct pass of UNION is unnecessary. Here five CSE rows plus five Mechanical rows yield ten rows.

2.9 INTERSECT

INTERSECT keeps rows that appear in **both** results. Entire rows must match (roll_no, student_name, and city). Amit Verma / Delhi and Sneha Reddy / Hyderabad were inserted into both tables, so the intersection has two rows. SQLite and SQL Server both support INTERSECT.

3. Schema

Three tables. employee holds eight attributes for questions 1–13. sr_no is the employee number (primary key). manager stores the manager's name, or NULL for the top manager. cse and mechanical are compatible three-column student lists for questions 14–22.

Attribute	Type	Constraint	Role
sr_no	INTEGER	PRIMARY KEY	Employee number
employee_name	TEXT	NOT NULL	Employee name
job	TEXT	NOT NULL	Job title
manager	TEXT	NULL allowed	Manager name
hire_date	DATE	NOT NULL	Date of joining
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary
department_no	INTEGER	NOT NULL	Department number
city	TEXT	NOT NULL	Work city

cse and mechanical share the same three attributes so they can be combined with set operators:

Attribute	Type	Constraint	Role
roll_no	INTEGER	PRIMARY KEY	Student roll number
student_name	TEXT	NOT NULL	Student name
city	TEXT	NOT NULL	Home city

4. Questions

The lab sheet numbered both CREATE and INSERT as **1**; they are shown here as **1(a)** and **1(b)**. Question 3 wrote “distant”; the SQL keyword is DISTINCT. Employee number is stored as sr_no. Questions **14–22** are the CSE / Mechanical UNION, UNION ALL, and INTERSECT exercise: create both student tables, list their rows, then combine them.

1(a). Create a table employee with the following attributes:

- sr no
- employee name
- job
- manager
- hire date
- salary
- department no
- city

1(b). Insert 10 rows in the above mentioned table.

2. SELECT all the details from the employee table.

3. Select only DISTINCT department number from the employee table.
The lab sheet wrote "distant" for this question.
4. SELECT all department numbers from the employee table.
5. SELECT employee name, salary WHERE salary is > 30000 rs from the employee table.
6. SELECT employee name, hire date WHERE city is not Delhi from the employee table.
7. SELECT employee name, hire date WHERE city is either Delhi or Mumbai from the employee table.
8. SELECT employee number, employee name WHERE salary is BETWEEN 30000 rs and 50000 rs.
9. SELECT employee number, employee name WHERE salary is NOT BETWEEN 30000 and 50000 from the employee table.
10. SELECT all the details WHERE city is among the following: Delhi, Mumbai, Chennai, Bangalore.
11. SELECT all the details WHERE city is not Mumbai, Delhi or Chennai from the employee table.
12. SELECT all the details from the employee table and ORDER BY employee names in descending order.
13. Display the list of employees in ascending order from the employee table.
14. Create a table cse (CSE students) with the following attributes:
 - roll no
 - student name
 - city
15. Insert 5 rows in the cse table.
16. Create a table mechanical (Mechanical students) with the following attributes:
 - roll no
 - student name
 - city
17. Insert 5 rows in the mechanical table.
Two rows also appear in cse (Amit Verma / Delhi and Sneha Reddy / Hyderabad) so INTERSECT is non-empty.
18. SELECT all the details from the cse table.
19. SELECT all the details from the mechanical table.
20. SELECT the UNION of cse and mechanical.
21. SELECT the UNION ALL of cse and mechanical.
22. SELECT the INTERSECT of cse and mechanical.

5. Procedure

Drop employee, cse, and mechanical if they already exist so the script can be re-run. Then answer each lab question in order. City values on employee are stored in title case (Delhi, Mumbai, ...) and matched with that same casing. Set-operation results are ordered by roll_no. The SQL for each question:

Q. No.	SQL solution
1(a)	<pre>CREATE TABLE employee (sr_no INTEGER PRIMARY KEY, employee_name TEXT NOT NULL, job TEXT NOT NULL, manager TEXT, hire_date DATE NOT NULL, salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0), department_no INTEGER NOT NULL, city TEXT NOT NULL);</pre>
1(b)	<pre>INSERT INTO employee (sr_no, employee_name, job, manager, hire_date, salary, department_no, city) VALUES (1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00, 10, 'Delhi'), (2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00, 20, 'Bangalore'), (3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00, 30, 'Mumbai'), (4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00, 40, 'Chennai'), (5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00, 50, 'Hyderabad'), (6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00, 20, 'Bangalore'), (7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00, 30, 'Pune'), (8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00, 60, 'Delhi'), (9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00, 20, 'Mumbai'), (10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00, 40, 'Chennai');</pre>
2	<pre>SELECT * FROM employee;</pre>
3	<pre>SELECT DISTINCT department_no FROM employee;</pre>
4	<pre>SELECT department_no FROM employee;</pre>
5	<pre>SELECT employee_name, salary FROM employee WHERE salary > 30000;</pre>
6	<pre>SELECT employee_name, hire_date FROM employee WHERE city <> 'Delhi';</pre>
7	<pre>SELECT employee_name, hire_date FROM employee WHERE city = 'Delhi' OR city = 'Mumbai';</pre>
8	<pre>SELECT sr_no, employee_name FROM employee WHERE salary BETWEEN 30000 AND 50000;</pre>
9	<pre>SELECT sr_no, employee_name FROM employee WHERE salary NOT BETWEEN 30000 AND 50000;</pre>
10	<pre>SELECT * FROM employee WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');</pre>
11	<pre>SELECT * FROM employee WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');</pre>
12	<pre>SELECT * FROM employee ORDER BY employee_name DESC;</pre>
13	<pre>SELECT * FROM employee ORDER BY employee_name ASC;</pre>
14	<pre>CREATE TABLE cse (roll_no INTEGER PRIMARY KEY, student_name TEXT NOT NULL, city TEXT NOT NULL);</pre>
15	<pre>INSERT INTO cse (roll_no, student_name, city) VALUES (1, 'Amit Verma', 'Delhi'), (2, 'Rahul Das', 'Kolkata'), (3, 'Sneha Reddy', 'Hyderabad'), (4, 'Isha Kapoor', 'Chandigarh'), (5, 'Dev Patel', 'Ahmedabad');</pre>

Q. No.	SQL solution
16	CREATE TABLE mechanical (roll_no INTEGER PRIMARY KEY, student_name TEXT NOT NULL, city TEXT NOT NULL);
17	INSERT INTO mechanical (roll_no, student_name, city) VALUES (1, 'Amit Verma', 'Delhi'), (3, 'Sneha Reddy', 'Hyderabad'), (6, 'Mohit Jain', 'Jaipur'), (7, 'Kavya Menon', 'Kochi'), (8, 'Tushar Rao', 'Nagpur');
18	SELECT * FROM cse;
19	SELECT * FROM mechanical;
20	SELECT * FROM cse UNION SELECT * FROM mechanical ORDER BY roll_no;
21	SELECT * FROM cse UNION ALL SELECT * FROM mechanical ORDER BY roll_no;
22	SELECT * FROM cse INTERSECT SELECT * FROM mechanical ORDER BY roll_no;

6. Source Code

SQLite script used in the lab compiler (09-09-2026/employee.sql). For the class SQL Server, run employee.sqlserver.sql.

```
-- =====
-- DBMS Lab · 09-09-2026
-- Employee table: CREATE, seed 10 rows, then SELECT queries
-- DISTINCT, WHERE, BETWEEN, IN, ORDER BY
-- Then CSE / Mechanical student tables: UNION, UNION ALL, INTERSECT
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
--
-- Lab questions (1(a)/1(b) = sheet's two "1." items; "distant" = DISTINCT)
-- 1(a). Create a table employee with the following attributes:
--      sr no, employee name, job, manager, hire date, salary,
--      department no, city
-- 1(b). Insert 10 rows in the above mentioned table
-- 2. SELECT all the details from the employee table
-- 3. Select only DISTINCT department number from employee table
--      (lab sheet wrote "distant")
-- 4. SELECT all department numbers from the employee table
-- 5. SELECT employee name, salary WHERE salary is > 30000 rs
--      from the employee table
-- 6. SELECT employee name, hire date WHERE city is not Delhi
--      from the employee table
-- 7. SELECT employee name, hire date WHERE city is either
--      Delhi or Mumbai from the employee table
-- 8. SELECT employee number, employee name WHERE salary is
--      BETWEEN 30000 rs and 50000 rs
-- 9. SELECT employee number, employee name WHERE salary is
--      NOT BETWEEN 30000 and 50000 from the employee table
-- 10. SELECT all the details WHERE city is among the following:
--      Delhi, Mumbai, Chennai, Bangalore
-- 11. SELECT all the details WHERE city is not Mumbai, Delhi
--      or Chennai from the employee table
-- 12. SELECT all the details from the employee table and
--      ORDER BY employee names in descending order
-- 13. Display the list of employees in ascending order from
--      the employee table.
-- 14. Create a table cse (CSE students) with roll no, student name, city
-- 15. Insert 5 rows in the cse table
-- 16. Create a table mechanical (Mechanical students) with the same attributes
-- 17. Insert 5 rows in the mechanical table
--      (two rows also appear in cse so INTERSECT is non-empty)
-- 18. SELECT all the details from the cse table
-- 19. SELECT all the details from the mechanical table
-- 20. SELECT the UNION of cse and mechanical
```

```

-- 21. SELECT the UNION ALL of cse and mechanical
-- 22. SELECT the INTERSECT of cse and mechanical
-- =====

DROP TABLE IF EXISTS employee;

-- -----
-- 1(a). Create a table employee with the following attributes:
--      sr no, employee name, job, manager, hire date, salary,
--      department no, city
-- -----
CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    manager        TEXT,
    hire_date      DATE NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    department_no  INTEGER NOT NULL,
    city           TEXT NOT NULL
);

-- -----
-- 1(b). Insert 10 rows in the above mentioned table
--      • salaries below, inside, and above 30000-50000
--      • cities: Delhi, Mumbai, Chennai, Bangalore, Hyderabad, Pune
--      • mixed department numbers (some duplicates)
--      • Rajesh Kumar is the top manager (manager IS NULL)
-- -----
INSERT INTO employee
(sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
VALUES
(1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00, 10, 'Delhi'),
(2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00, 20, 'Bangalore'),
(3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00, 30, 'Mumbai'),
(4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00, 40, 'Chennai'),
(5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00, 50, 'Hyderabad'),
(6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00, 20, 'Bangalore'),
(7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00, 30, 'Pune'),
(8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00, 60, 'Delhi'),
(9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00, 20, 'Mumbai'),
(10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00, 40, 'Chennai');

-- -----
-- 2. select all the details from the employee table
-- -----
-- All the details from employee
SELECT * FROM employee;

-- -----
-- 3. Select only DISTINCT department number from employee table
--      (lab sheet wrote "distant")
-- -----
-- Distant (DISTINCT) department number
SELECT DISTINCT department_no FROM employee;

-- -----
-- 4. select all department numbers from employee table
-- -----
-- All department numbers
SELECT department_no FROM employee;

-- -----
-- 5. select employee name, salary where salary is >30000 rs
--      from employee table
-- -----
-- Salary > 30000 rs
SELECT employee_name, salary
FROM employee
WHERE salary > 30000;

-- -----
-- 6. select employee name, hire date where city is not delhi
--      from employee table
-- -----
-- City is not delhi
SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';

-- -----
-- 7. select employee name, hire date where city is either
--      delhi or mumbai from employee table
-- -----
-- City is either delhi or mumbai
SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';
-- -----

```

```

-- 8. select employee number, employee name where salary is
--    between 30000 rs and 50000 rs
-----
-- Salary between 30000 rs and 50000 rs
SELECT sr_no, employee_name
FROM employee
WHERE salary BETWEEN 30000 AND 50000;

-----
-- 9. select employee number, employee name where salary is
--    not between 30000 and 50000 from employee table
-----
-- Salary not between 30000 and 50000
SELECT sr_no, employee_name
FROM employee
WHERE salary NOT BETWEEN 30000 AND 50000;

-----
-- 10. select all the details where city is among the following:
--     delhi, mumbai, chennai, Bangalore
-----
-- City among delhi, mumbai, chennai, Bangalore
SELECT *
FROM employee
WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');

-----
-- 11. select all the details where city is not mumbai, delhi
--     or chennai from employee table
-----
-- City is not mumbai, delhi or chennai
SELECT *
FROM employee
WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');

-----
-- 12. select all the details from the employee table and
--     order by employee names in descending order.
-----
-- Order by employee names descending
SELECT *
FROM employee
ORDER BY employee_name DESC;

-----
-- 13. display the list of employees in ascending order from
--     employee table.
-----
-- Display employees in ascending order
SELECT *
FROM employee
ORDER BY employee_name ASC;

-----
-- =====
-- CSE / Mechanical student tables – UNION, UNION ALL, INTERSECT
-- Compatible columns (roll_no, student_name, city). Two identical
-- rows appear in both tables so INTERSECT returns a non-empty set.
-- =====

DROP TABLE IF EXISTS cse;
DROP TABLE IF EXISTS mechanical;

-----
-- 14. Create a table cse (CSE students) with the following
--     attributes: roll no, student name, city
-----
CREATE TABLE cse (
    roll_no      INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL,
    city         TEXT NOT NULL
);

-----
-- 15. Insert 5 rows in the cse table
-----
INSERT INTO cse (roll_no, student_name, city)
VALUES
    (1, 'Amit Verma', 'Delhi'),
    (2, 'Rahul Das', 'Kolkata'),
    (3, 'Sneha Reddy', 'Hyderabad'),
    (4, 'Isha Kapoor', 'Chandigarh'),
    (5, 'Dev Patel', 'Ahmedabad');

-----
-- 16. Create a table mechanical (Mechanical students) with the
--     following attributes: roll no, student name, city
-----
CREATE TABLE mechanical (
    roll_no      INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL,

```



```

        city          TEXT NOT NULL
    );

-- -----
-- 17. Insert 5 rows in the mechanical table
--      Two rows match cse (Amit Verma / Delhi, Sneha Reddy / Hyderabad)
-- -----
INSERT INTO mechanical (roll_no, student_name, city)
VALUES
    (1, 'Amit Verma', 'Delhi'),
    (3, 'Sneha Reddy', 'Hyderabad'),
    (6, 'Mohit Jain', 'Jaipur'),
    (7, 'Kavya Menon', 'Kochi'),
    (8, 'Tushar Rao', 'Nagpur');

-- -----
-- 18. SELECT all the details from the cse table
-- -----
-- All the details from CSE
SELECT * FROM cse;

-- -----
-- 19. SELECT all the details from the mechanical table
-- -----
-- All the details from Mechanical
SELECT * FROM mechanical;

-- -----
-- 20. SELECT the UNION of cse and mechanical
--      (unique rows from either table)
-- -----
-- UNION of CSE and Mechanical
SELECT * FROM cse
UNION
SELECT * FROM mechanical
ORDER BY roll_no;

-- -----
-- 21. SELECT the UNION ALL of cse and mechanical
--      (all rows, including duplicates)
-- -----
-- UNION ALL of CSE and Mechanical
SELECT * FROM cse
UNION ALL
SELECT * FROM mechanical
ORDER BY roll_no;

-- -----
-- 22. SELECT the INTERSECT of cse and mechanical
--      (rows that appear in both tables)
-- -----
-- INTERSECT of CSE and Mechanical
SELECT * FROM cse
INTERSECT
SELECT * FROM mechanical
ORDER BY roll_no;

```

7. Output

The script was executed in SQLite in memory. CREATE and INSERT print a status line; each SELECT prints its result set. NULL in the manager column is shown as an empty cell.

```

SQL> DROP TABLE IF EXISTS employee;
-- OK

SQL> CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    manager        TEXT,
    hire_date      DATE NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    department_no  INTEGER NOT NULL,
    city           TEXT NOT NULL
);
-- OK

SQL> INSERT INTO employee
(sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
VALUES
    (1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00,
10, 'Delhi'),
    (2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00,
20, 'Bangalore'),
    (3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00,
30, 'Mumbai'),
    (4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00,
40, 'Chennai'),
    (5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00,
50, 'Hyderabad'),

```

```

        (6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00,
20, 'Bangalore'),
        (7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00,
30, 'Pune'),
        (8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00,
60, 'Delhi'),
        (9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00,
20, 'Mumbai'),
        (10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00,
40, 'Chennai');
-- 10 row(s) inserted

```

```

SQL> SELECT * FROM employee;
sr_no  employee_name  job              manager      hire_date  salary  department_no  city
-----
1      Rajesh Kumar   General Manager
2      Ananya Sharma  Software Engineer Rajesh Kumar  2021-07-01 45000   20             Bangalore
3      Rohan Mehta    Accountant       Rajesh Kumar  2020-01-15 28000   30             Mumbai
4      Priya Nair     HR Executive     Rajesh Kumar  2022-04-20 32000   40             Chennai
5      Vikram Singh   Sales Manager    Rajesh Kumar  2019-11-08 52000   50             Hyderabad
6      Fatima Khan    Software Engineer Ananya Sharma 2023-02-14 38000   20             Bangalore
7      Arjun Patel    Clerk            Rohan Mehta  2024-06-01 18000   30             Pune
8      Neha Gupta     Marketing Analyst Rajesh Kumar  2021-09-10 35000   60             Delhi
9      Karan Iyer     Database Admin   Ananya Sharma 2020-12-05 48000   20             Mumbai
10     Meera Joshi    Receptionist     Priya Nair    2023-08-22 22000   40             Chennai

```

```

SQL> SELECT DISTINCT department_no FROM employee;
department_no
-----
10
20
30
40
50
60

```

```

SQL> SELECT department_no FROM employee;
department_no
-----
10
20
30
40
50
20
30
60
20
40

```

```

SQL> SELECT employee_name, salary
FROM employee
WHERE salary > 30000;
employee_name  salary
-----
Rajesh Kumar   85000
Ananya Sharma  45000
Priya Nair     32000
Vikram Singh   52000
Fatima Khan    38000
Neha Gupta     35000
Karan Iyer     48000

```

```

SQL> SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';
employee_name  hire_date
-----
Ananya Sharma  2021-07-01
Rohan Mehta    2020-01-15
Priya Nair     2022-04-20
Vikram Singh   2019-11-08
Fatima Khan    2023-02-14
Arjun Patel    2024-06-01
Karan Iyer     2020-12-05
Meera Joshi    2023-08-22

```

```

SQL> SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';
employee_name  hire_date
-----
Rajesh Kumar   2018-03-12
Rohan Mehta    2020-01-15
Neha Gupta     2021-09-10
Karan Iyer     2020-12-05

```

```

SQL> SELECT sr_no, employee_name
FROM employee
WHERE salary BETWEEN 30000 AND 50000;
sr_no  employee_name
-----
2      Ananya Sharma
4      Priya Nair
6      Fatima Khan
8      Neha Gupta
9      Karan Iyer

```

```

SQL> SELECT sr_no, employee_name
FROM employee
WHERE salary NOT BETWEEN 30000 AND 50000;
sr_no  employee_name

```

```

-----
1    Rajesh Kumar
3    Rohan Mehta
5    Vikram Singh
7    Arjun Patel
10   Meera Joshi

```

```

SQL> SELECT *
      FROM employee
     WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');
sr_no  employee_name  job              manager      hire_date  salary  department_no  city
-----
1      Rajesh Kumar   General Manager  Rajesh Kumar  2018-03-12  85000   10             Delhi
2      Ananya Sharma  Software Engineer Rajesh Kumar  2021-07-01  45000   20             Bangalore
3      Rohan Mehta    Accountant       Rajesh Kumar  2020-01-15  28000   30             Mumbai
4      Priya Nair      HR Executive     Rajesh Kumar  2022-04-20  32000   40             Chennai
6      Fatima Khan    Software Engineer Ananya Sharma 2023-02-14  38000   20             Bangalore
8      Neha Gupta     Marketing Analyst Rajesh Kumar  2021-09-10  35000   60             Delhi
9      Karan Iyer     Database Admin   Ananya Sharma 2020-12-05  48000   20             Mumbai
10     Meera Joshi    Receptionist     Priya Nair    2023-08-22  22000   40             Chennai

```

```

SQL> SELECT *
      FROM employee
     WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');
sr_no  employee_name  job              manager      hire_date  salary  department_no  city
-----
2      Ananya Sharma  Software Engineer Rajesh Kumar  2021-07-01  45000   20             Bangalore
5      Vikram Singh    Sales Manager     Rajesh Kumar  2019-11-08  52000   50             Hyderabad
6      Fatima Khan    Software Engineer Ananya Sharma 2023-02-14  38000   20             Bangalore
7      Arjun Patel     Clerk             Rohan Mehta   2024-06-01  18000   30             Pune

```

```

SQL> SELECT *
      FROM employee
     ORDER BY employee_name DESC;
sr_no  employee_name  job              manager      hire_date  salary  department_no  city
-----
5      Vikram Singh    Sales Manager     Rajesh Kumar  2019-11-08  52000   50             Hyderabad
3      Rohan Mehta    Accountant       Rajesh Kumar  2020-01-15  28000   30             Mumbai
1      Rajesh Kumar   General Manager  Rajesh Kumar  2018-03-12  85000   10             Delhi
4      Priya Nair      HR Executive     Rajesh Kumar  2022-04-20  32000   40             Chennai
8      Neha Gupta     Marketing Analyst Rajesh Kumar  2021-09-10  35000   60             Delhi
10     Meera Joshi    Receptionist     Priya Nair    2023-08-22  22000   40             Chennai
9      Karan Iyer     Database Admin   Ananya Sharma 2020-12-05  48000   20             Mumbai
6      Fatima Khan    Software Engineer Ananya Sharma 2023-02-14  38000   20             Bangalore
7      Arjun Patel     Clerk             Rohan Mehta   2024-06-01  18000   30             Pune
2      Ananya Sharma  Software Engineer Rajesh Kumar  2021-07-01  45000   20             Bangalore

```

```

SQL> SELECT *
      FROM employee
     ORDER BY employee_name ASC;
sr_no  employee_name  job              manager      hire_date  salary  department_no  city
-----
2      Ananya Sharma  Software Engineer Rajesh Kumar  2021-07-01  45000   20             Bangalore
7      Arjun Patel     Clerk             Rohan Mehta   2024-06-01  18000   30             Pune
6      Fatima Khan    Software Engineer Ananya Sharma 2023-02-14  38000   20             Bangalore
9      Karan Iyer     Database Admin   Ananya Sharma 2020-12-05  48000   20             Mumbai
10     Meera Joshi    Receptionist     Priya Nair    2023-08-22  22000   40             Chennai
8      Neha Gupta     Marketing Analyst Rajesh Kumar  2021-09-10  35000   60             Delhi
4      Priya Nair      HR Executive     Rajesh Kumar  2022-04-20  32000   40             Chennai
1      Rajesh Kumar   General Manager  Rajesh Kumar  2018-03-12  85000   10             Delhi
3      Rohan Mehta    Accountant       Rajesh Kumar  2020-01-15  28000   30             Mumbai
5      Vikram Singh    Sales Manager     Rajesh Kumar  2019-11-08  52000   50             Hyderabad

```

```

SQL> DROP TABLE IF EXISTS cse;
-- OK

```

```

SQL> DROP TABLE IF EXISTS mechanical;
-- OK

```

```

SQL> CREATE TABLE cse (
    roll_no      INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL,
    city         TEXT NOT NULL
);
-- OK

```

```

SQL> INSERT INTO cse (roll_no, student_name, city)
VALUES
    (1, 'Amit Verma', 'Delhi'),
    (2, 'Rahul Das', 'Kolkata'),
    (3, 'Sneha Reddy', 'Hyderabad'),
    (4, 'Isha Kapoor', 'Chandigarh'),
    (5, 'Dev Patel', 'Ahmedabad');
-- 5 row(s) inserted

```

```

SQL> CREATE TABLE mechanical (
    roll_no      INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL,
    city         TEXT NOT NULL
);
-- OK

```

```

SQL> INSERT INTO mechanical (roll_no, student_name, city)
VALUES
    (1, 'Amit Verma', 'Delhi'),
    (3, 'Sneha Reddy', 'Hyderabad'),
    (6, 'Mohit Jain', 'Jaipur'),
    (7, 'Kavya Menon', 'Kochi'),
    (8, 'Tushar Rao', 'Nagpur');
-- 5 row(s) inserted

```

```

SQL> SELECT * FROM cse;

```

roll_no	student_name	city
1	Amit Verma	Delhi
2	Rahul Das	Kolkata
3	Sneha Reddy	Hyderabad
4	Isha Kapoor	Chandigarh
5	Dev Patel	Ahmedabad

```
SQL> SELECT * FROM mechanical;
roll_no student_name city
-----
1       Amit Verma   Delhi
3       Sneha Reddy  Hyderabad
6       Mohit Jain     Jaipur
7       Kavya Menon   Kochi
8       Tushar Rao    Nagpur
```

```
SQL> SELECT * FROM cse
UNION
SELECT * FROM mechanical
ORDER BY roll_no;
roll_no student_name city
-----
1       Amit Verma   Delhi
2       Rahul Das    Kolkata
3       Sneha Reddy  Hyderabad
4       Isha Kapoor  Chandigarh
5       Dev Patel    Ahmedabad
6       Mohit Jain   Jaipur
7       Kavya Menon  Kochi
8       Tushar Rao   Nagpur
```

```
SQL> SELECT * FROM cse
UNION ALL
SELECT * FROM mechanical
ORDER BY roll_no;
roll_no student_name city
-----
1       Amit Verma   Delhi
1       Amit Verma   Delhi
2       Rahul Das    Kolkata
3       Sneha Reddy  Hyderabad
3       Sneha Reddy  Hyderabad
4       Isha Kapoor  Chandigarh
5       Dev Patel    Ahmedabad
6       Mohit Jain   Jaipur
7       Kavya Menon  Kochi
8       Tushar Rao   Nagpur
```

```
SQL> SELECT * FROM cse
INTERSECT
SELECT * FROM mechanical
ORDER BY roll_no;
roll_no student_name city
-----
1       Amit Verma   Delhi
3       Sneha Reddy  Hyderabad
```

8. Results

Ten employees load successfully, then five CSE students and five Mechanical students. Each lab question returns a non-empty result on this seed data:

Q. No.	Rows	What the result shows
1(a)	—	Table employee created with 8 attributes
1(b)	10	10 rows inserted
2	10	All details from the employee table
3	6	DISTINCT department numbers: 10, 20, 30, 40, 50, 60
4	10	All department numbers (dept 20 appears three times)
5	7	Employee name and salary where salary is > 30000 rs
6	8	Employee name and hire date where city is not Delhi
7	4	Employee name and hire date where city is Delhi or Mumbai
8	5	Employee number and name where salary is between 30000 rs and 50000 rs
9	5	Employee number and name where salary is not between 30000 and 50000
10	8	All details where city is Delhi, Mumbai, Chennai, or Bangalore
11	4	All details where city is not Mumbai, Delhi, or Chennai
12	10	All details ordered by employee names descending
13	10	List of employees in ascending order
14	—	Table cse created (roll no, student name, city)
15	5	5 CSE students inserted
16	—	Table mechanical created with the same attributes
17	5	5 Mechanical students inserted (2 rows also in cse)
18	5	All details from the cse table
19	5	All details from the mechanical table
20	8	UNION — unique rows from either table (5 + 5 – 2)
21	10	UNION ALL — all rows including the 2 duplicates
22	2	INTERSECT — Amit Verma / Delhi and Sneha Reddy / Hyderabad

Question 3 (DISTINCT) collapses ten department values to six unique numbers; question 4 lists all ten. Questions 8 and 9 partition salaries with no overlap ($5 + 5 = 10$). Question 11 keeps Bangalore, Hyderabad, and Pune — cities that are not Mumbai, Delhi, or Chennai. Questions 20–22 show the set-operator identities on this seed: UNION has 8 rows, UNION ALL has 10, and INTERSECT has the 2 shared students.

9. Conclusion

A single SELECT can project columns, remove duplicates, filter with comparison and set predicates, and sort the result. DISTINCT answers “which unique values exist?” WHERE answers “which rows match this condition?” BETWEEN and IN express ranges and lists clearly. ORDER BY changes presentation only. UNION / UNION ALL / INTERSECT combine two compatible queries: unique rows from either side, every row including duplicates, or only rows common to both. Together these clauses are the core of everyday retrieval in SQL.